LAB 2 — Aplicación desacoplada

Integración y mensajería: productor y consumidor sin dependencia temporal

Computación en Nube · EAFIT · Sesión 10 · 22 de septiembre de 2026

Duración	Modalidad	Entorno	Carácter
100–110 minutos	Equipos de 4	Cuenta AWS personal	Evaluativo · 10 %

1. Propósito y resultado

Implementar un flujo simple de Digital Café Luna donde un productor genere trabajo, Amazon SQS lo conserve temporalmente y un consumidor lo procese cuando esté disponible. La práctica debe demostrar que productor y consumidor no necesitan ejecutarse al mismo tiempo.

Pregunta rectora: ¿Cómo desacoplamos la generación de trabajo de su procesamiento para que una indisponibilidad temporal del consumidor no detenga al productor?



Figura 1. Arquitectura de LAB 2. Productor y consumidor son procesos independientes dentro de una sola EC2; SQS desacopla temporalmente el envío y el procesamiento.

> **OJO TÉCNICO.** Una sola EC2 permite demostrar desacoplamiento lógico/temporal, pero no aislamiento ante una falla de instancia. Si la EC2 falla, productor y consumidor dejan de ejecutarse. Ese no es el objetivo de este laboratorio.

Objetivos de aprendizaje

- Distinguir una interacción síncrona de un flujo asíncrono mediado por una cola.
- Explicar las responsabilidades de productor, cola y consumidor.
- Demostrar backlog cuando el consumidor está detenido y procesamiento posterior al reactivarlo.
- Relacionar visibility timeout, retry y DeleteMessage con procesamiento exitoso.
- Reconocer at-least-once como propiedad que exige tolerar posibles entregas repetidas.

2. Baseline, recursos y convenciones

Reutilice únicamente infraestructura realmente disponible y necesaria. LAB 2 no depende técnicamente de una persistencia implementada en S09. La práctica utiliza una sola EC2 en us-east-1, con salida a Internet/AWS APIs, y una cola SQS Standard.

Etiquetas sugeridas: dcl:project=digital-cafe-luna, dcl:environment=dev, dcl:owner-team=team-XX y dcl:managed-by=console.

Tipo	Recurso	Nombre / configuración
Reutilizar	VPC / subnet pública	Baseline disponible · us-east-1 · salida a Internet
Crear	Security Group	dcl-dev-sg-lab2 · SSH/22 solo desde My IP

Tipo	Recurso	Nombre / configuración
Crear	IAM Role / Instance Profile	dcl-dev-role-lab2-sqs
Crear	EC2	dcl-dev-lab2-worker · Amazon Linux 2023 · tipo pequeño elegible
Crear	SQS Standard	dcl-dev-orders-queue
Crear en EC2	Productor	producer.py
Crear en EC2	Consumidor	consumer.py
Opcional	DLQ	Solo extensión; no criterio principal de aceptación

OJO TÉCNICO. No almacene Access Key ID ni Secret Access Key en los scripts. Boto3 debe obtener credenciales temporales del IAM Role asociado a la EC2 mediante su Instance Profile.

3. Paso a paso en la consola

Fase 0 — Descubrir y verificar el entorno

Qué hacemos: Confirmar región, red disponible y una subnet desde la cual la EC2 pueda alcanzar los endpoints públicos de AWS.

Por qué: El laboratorio debe partir del estado real y crear únicamente lo necesario para demostrar mensajería.

1. Confirme la Región us-east-1.

2. Identifique una VPC y subnet pública disponible. Verifique que la subnet tenga ruta 0.0.0.0/0 hacia un Internet Gateway y que la EC2 pueda recibir IPv4 pública.

3. No reconstruya recursos de S08/S09 que no sean necesarios para este flujo.

Checkpoint: Existe una ubicación válida para una EC2 con conectividad de salida hacia AWS APIs.

Fase 1 — Crear la cola SQS Standard

Qué hacemos: Crear el intermediario que conservará trabajo mientras el consumidor no esté disponible.

Por qué: La cola permite separar temporalmente la producción del procesamiento.

1. Abra Amazon SQS > Create queue.

2. Seleccione Type = Standard y Name = dcl-dev-orders-queue.

3. Configure Visibility timeout = 30 seconds.

4. Configure Receive message wait time = 5 seconds.

5. Mantenga los demás parámetros en valores predeterminados salvo indicación del docente. Cree la cola.

6. Registre Queue URL y ARN.

Validación: La cola existe, es Standard, VisibilityTimeout = 30 s y ReceiveMessageWaitTimeSeconds = 5 s.

> **OJO TÉCNICO.** SQS Standard ofrece entrega at-least-once y orden best-effort. No use el orden de llegada como criterio de éxito del laboratorio.

Fase 2 — Crear IAM Role / Instance Profile

Qué hacemos: Autorizar a la aplicación de la EC2 para operar únicamente sobre la cola del laboratorio.

Por qué: Las aplicaciones en EC2 deben usar credenciales temporales del role, no credenciales estáticas dentro del código.

1. Cree un IAM role para trusted entity = AWS service > EC2. Nombre sugerido: dcl-dev-role-lab2-sqs.

2. Cree/asocie una policy de mínimo privilegio reemplazando ACCOUNT_ID por el ID de su cuenta:

```
{
 "Version": "2012-10-17",
 "Statement": [{
  "Effect": "Allow",
  "Action": [
   "sqs:GetQueueAttributes",
   "sqs:SendMessage",
   "sqs:ReceiveMessage",
   "sqs:DeleteMessage"
  ],
  "Resource": "arn:aws:sqs:us-east-1:ACCOUNT_ID:dcl-dev-orders-queue"
 }]
}
```

3. Confirme que el role queda disponible mediante un Instance Profile para asociarlo a EC2.

Validación: El role no contiene permisos amplios sobre SQS y no existen access keys destinadas a los scripts.

Fase 3 — Lanzar una EC2 para ambos procesos

Qué hacemos: Crear una única unidad de cómputo que ejecutará productor y consumidor como procesos separados.

Por qué: Queremos observar independencia temporal entre procesos sin agregar distribución física innecesaria.

1. Abra EC2 > Instances > Launch instances. Name: dcl-dev-lab2-worker.

2. Seleccione Amazon Linux 2023 y un instance type pequeño elegible en su cuenta.

3. Seleccione la VPC/subnet validada y habilite Auto-assign public IP.

4. Asocie dcl-dev-role-lab2-sqs como IAM instance profile.

5. Use un key pair disponible para el equipo. En el Security Group permita SSH TCP/22 únicamente desde My IP; nunca use 0.0.0.0/0 para esta práctica.

6. Mantenga salida HTTPS/443 permitida hacia AWS APIs mediante el egress predeterminado.

7. Lance la instancia y conéctese como ec2-user.

Validación: La EC2 está Running, tiene el role asociado y puede alcanzar AWS APIs.

Fase 4 — Preparar el entorno y verificar identidad

Qué hacemos: Confirmar que Python 3.11/Boto3 pueden usar las credenciales temporales del Instance Profile.

Por qué: Amazon Linux 2023 conserva /usr/bin/python3 en Python 3.9. Instalaremos Python 3.11 en paralelo y lo invocaremos explícitamente, sin cambiar el symlink del Python del sistema.

```
aws sts get-caller-identity
sudo dnf install -y python3.11 python3.11-pip
python3.11 --version
python3.11 -m pip install --user boto3
```

Guarde la URL de la cola como variable de entorno. Sustituya el valor por su Queue URL real:

```
export AWS_REGION=us-east-1
export QUEUE_URL="https://sqs.us-east-1.amazonaws.com/ACCOUNT_ID/dcl-dev-orders-queue"
```

Validación: aws sts get-caller-identity devuelve la identidad del role asociado a la EC2.

Fase 5 — Crear producer.py

Qué hacemos: Crear un productor mínimo que envíe pedidos identificables a SQS.

Por qué: El productor debe terminar correctamente aunque consumer.py no esté ejecutándose.

```
cat > producer.py <<'PY'
import boto3, json, os, sys
from datetime import datetime, timezone


queue_url = os.environ["QUEUE_URL"]
region = os.getenv("AWS_REGION", "us-east-1")
sqs = boto3.client("sqs", region_name=region)


order_id = sys.argv[1] if len(sys.argv) > 1 else "DCL-001"
simulate_failure = "--fail" in sys.argv
body = {
    "orderId": order_id,
    "type": "ORDER_CREATED",
    "createdAt": datetime.now(timezone.utc).isoformat(),
    "simulateFailure": simulate_failure
}


response = sqs.send_message(QueueUrl=queue_url, MessageBody=json.dumps(body))
print(f"SENT orderId={order_id} MessageId={response['MessageId']} fail={simulate_failure}")
PY
```

Prueba sintáctica:

```
python3.11 -m py_compile producer.py
```

Validación: El archivo compila sin error.

Fase 6 — Crear consumer.py

Qué hacemos: Crear un consumidor que recibe, procesa y elimina mensajes solo después del éxito.

Por qué: ReceiveMessage no significa que el trabajo esté completado. DeleteMessage representa la confirmación de procesamiento exitoso.

```
cat > consumer.py <<'PY'
import boto3, json, os, time


queue_url = os.environ["QUEUE_URL"]
region = os.getenv("AWS_REGION", "us-east-1")
sqs = boto3.client("sqs", region_name=region)


print("Consumer ON. Ctrl+C para detener.")
while True:
  response = sqs.receive_message(
    QueueUrl=queue_url,
    MaxNumberOfMessages=1,
    WaitTimeSeconds=5,
    VisibilityTimeout=30,
    AttributeNames=["ApproximateReceiveCount"]
  )
  for message in response.get("Messages", []):
    body = json.loads(message["Body"])
    count = message.get("Attributes", {}).get("ApproximateReceiveCount", "?")
    print(f"RECEIVED MessageId={message['MessageId']} orderId={body.get('orderId')} receiveCount={count}")


    if body.get("simulateFailure"):
      print("PROCESSING FAILED — message NOT deleted")
      continue


    time.sleep(2)
    print(f"PROCESSED orderId={body.get('orderId')}")
    sqs.delete_message(QueueUrl=queue_url, ReceiptHandle=message["ReceiptHandle"])
    print(f"DELETED MessageId={message['MessageId']}")
PY
```

Prueba sintáctica:

```
python3.11 -m py_compile consumer.py
```

Validación: El archivo compila y WaitTimeSeconds = 5 / VisibilityTimeout = 30 están explícitos.

Fase 7 — Happy path

Qué hacemos: Verificar primero el flujo normal de extremo a extremo.

Por qué: Necesitamos un baseline funcional antes de detener el consumidor o provocar fallos.

1. En una terminal, inicie el consumidor:

```
python3.11 consumer.py
```

2. En una segunda terminal sobre la misma EC2, exporte AWS_REGION y QUEUE_URL y envíe un pedido:

```
python3.11 producer.py DCL-001
```

3. Observe en consumer.py: RECEIVED → PROCESSED → DELETED.

Validación: DCL-001 se procesa y se elimina. La cola vuelve a no tener mensajes visibles.

Fase 8 — Experimento central: consumer OFF → backlog → consumer ON

Qué hacemos: Detener solo el consumidor, mantener la EC2 activa y continuar generando trabajo.

Por qué: Esta es la evidencia principal de desacoplamiento temporal/lógico.

1. En la terminal del consumidor, presione Ctrl+C. La EC2 permanece Running.
2. Ejecute desde la otra terminal:

```
python3.11 producer.py DCL-002
python3.11 producer.py DCL-003
python3.11 producer.py DCL-004
```

3. Verifique el backlog desde SQS > Monitoring/Send and receive messages o mediante CLI:

```
aws sqs get-queue-attributes   --queue-url "$QUEUE_URL"   --attribute-names ApproximateNumberOfMessages
ApproximateNumberOfMessagesNotVisible   --query 'Attributes'   --output table
```

Checkpoint obligatorio: consumer = OFF, producer = OK y backlog observado = 3 mensajes visibles.

4. Reactive el consumidor:

```
python3.11 consumer.py
```

5. Observe el procesamiento de DCL-002, DCL-003 y DCL-004. El orden exacto no es criterio de aceptación.
6. Repita la consulta de atributos.

Validación: El backlog final observado es 0 y los tres pedidos fueron procesados.

> **OJO TÉCNICO.** Los atributos ApproximateNumberOfMessages* son aproximados. Úselos como evidencia operativa junto con la salida de producer.py/consumer.py, no como un contador transaccional exacto.

Fase 9 — Fallo controlado, visibility timeout y retry

Qué hacemos: Enviar un mensaje que el consumidor recibe pero deliberadamente no elimina.

Por qué: Queremos observar que un mensaje recibido permanece en la cola, queda temporalmente invisible y puede reaparecer para reintento.

1. Con consumer.py activo, envíe:

```
python3.11 producer.py DCL-FAIL-001 --fail
```

2. Registre el MessageId mostrado por producer.py.
3. Observe en consumer.py: RECEIVED y PROCESSING FAILED — message NOT deleted.

4. Consulte los atributos de la cola inmediatamente. El mensaje puede aparecer como no visible/in-flight.

5. Espere aproximadamente 30 segundos desde la recepción. Observe que el mismo MessageId vuelve a ser recibido. El ReceiptHandle puede cambiar en cada recepción.

6. Detenga consumer.py después de confirmar el retry para evitar un ciclo indefinido de reintentos durante la práctica.

Validación: El mismo MessageId reaparece después de expirar el visibility timeout y el retry queda confirmado.

> **OJO TÉCNICO.** DeleteMessage utiliza el ReceiptHandle de la recepción actual, no el MessageId. En SQS Standard puede ocurrir más de una entrega; diseñar consumidores idempotentes es una consideración básica, aunque la idempotencia completa no se implementa en LAB 2.

4. Verificación con AWS CLI

Estos comandos complementan las evidencias funcionales. Ejecútelos desde la EC2 con AWS_REGION y QUEUE_URL definidos.

```
# Identidad temporal del Instance Profile
aws sts get-caller-identity
```

```
# Configuración congelada de la cola
aws sqs get-queue-attributes   --queue-url "$QUEUE_URL"   --attribute-names VisibilityTimeout ReceiveMessageWaitTimeSeconds QueueArn
--query 'Attributes'   --output table
```

```
# Estado aproximado del backlog
aws sqs get-queue-attributes   --queue-url "$QUEUE_URL"   --attribute-names ApproximateNumberOfMessages
ApproximateNumberOfMessagesNotVisible   --query 'Attributes'   --output table
```

5. Evidencias y criterios de aceptación

Las evidencias deben demostrar comportamiento antes, durante y después; una captura de recursos creados no sustituye la validación funcional.

Evidencias mínimas:

- Arquitectura productor → SQS Standard → consumidor → resultado observable.
- Identidad de la EC2 usando IAM Role / Instance Profile, sin credenciales hardcoded.
- Happy path con MessageId enviado, recibido, procesado y eliminado.
- Consumer OFF con tres mensajes producidos y backlog = 3 observado.
- Consumer ON procesando el backlog y backlog final = 0.
- Fallo controlado mostrando NO DeleteMessage, mensaje no visible temporalmente y retry del mismo MessageId.
- Configuración VisibilityTimeout = 30 s y WaitTimeSeconds = 5 s.
- Cleanup final.

Comprobación	Resultado esperado
Arquitectura	Una EC2 ejecuta producer.py y consumer.py como procesos independientes; SQS media entre ambos.
IAM	La EC2 usa IAM Role / Instance Profile y los scripts no contienen access keys.
Happy path	SendMessage → ReceiveMessage → procesamiento → DeleteMessage tras éxito.
Desacoplamiento	Con consumer OFF, producer sigue enviando y se observa backlog = 3.
Recuperación del backlog	Con consumer ON, los pendientes se procesan y backlog final observado = 0.
Fallo / retry	El mensaje fallido no se elimina, queda no visible y el mismo MessageId reaparece tras ~30 s.
Polling	consumer.py usa WaitTimeSeconds = 5.
Semántica SQS	El equipo no asume orden estricto ni exactly-once; reconoce at-least-once.
Alcance	La práctica demuestra desacoplamiento temporal/lógico, no aislamiento ante falla de EC2.

6. Entregable evaluativo y rúbrica

Entregue un documento breve y trazable con los diez componentes comunes definidos para los laboratorios de Digital Café Luna:

1. Objetivo y arquitectura.
2. Recursos implementados.
3. Procedimiento resumido.
4. Validaciones.
5. Evidencias.
6. Problemas y soluciones.
7. Seguridad y costos.
8. Cleanup.
9. Contribuciones.
10. Retrospectiva.

Criterio	Peso	Evidencia principal en LAB 2
Implementación técnica	30 %	EC2 + IAM Role/Profile + SQS + producer.py + consumer.py funcionales.
Validación funcional	20 %	Happy path, backlog OFF/ON y retry demostrados.
Arquitectura y buenas prácticas	15 %	Desacoplamiento explicado; permisos mínimos; semántica SQS correcta.
Seguridad, costos y cleanup	10 %	Sin credenciales hardcoded; SSH restringido; limpieza completa.
Evidencias y documentación	15 %	Antes/durante/después y trazabilidad de MessageId/backlog.
Colaboración y defensa	10 %	Roles, contribuciones y explicación técnica por cualquier integrante.

7. Seguridad, costos y cleanup obligatorio

Seguridad mínima

- Use IAM Role / Instance Profile para credenciales temporales; no copie access keys a la EC2 ni al código.
- Restrinja la policy SQS a dcl-dev-orders-queue.
- Para acceso operativo, limite SSH/22 a My IP. No use 0.0.0.0/0.
- No haga pública la cola ni agregue permisos anónimos.

Cleanup

Capture primero todas las evidencias evaluativas. Luego elimine los recursos creados exclusivamente para LAB 2 en un orden que evite dependencias innecesarias:

1. Detenga consumer.py.
2. Elimine dcl-dev-lab2-worker y espere estado Terminated.
3. Elimine dcl-dev-orders-queue. Si creó una DLQ opcional, elimínela también.
4. Elimine dcl-dev-sg-lab2 cuando ya no tenga interfaces asociadas.
5. Desvincule/elimine la policy exclusiva y elimine dcl-dev-role-lab2-sqs / Instance Profile si no serán reutilizados.
6. Conserve únicamente infraestructura heredada que no haya sido creada específicamente para LAB 2.

Validación: No queda una EC2 del laboratorio en Running/Stopped, no queda la cola dcl-dev-orders-queue y no permanecen recursos temporales exclusivos sin propósito.

8. Extensión opcional — Dead-Letter Queue

La DLQ no forma parte del camino crítico ni de los criterios principales de aceptación. Si el tiempo lo permite, discuta o configure una DLQ para responder: ¿qué debería ocurrir con un mensaje que falla repetidamente y no puede procesarse correctamente?

No convierta esta extensión en un ejercicio de redrive avanzado. Su objetivo es reconocer cuándo separar mensajes problemáticos evita reintentos indefinidos sobre la cola principal.

9. Cierre técnico

- ¿Qué evidencia demuestra que el productor no depende de que el consumidor esté ejecutándose?
- ¿Por qué ReceiveMessage no significa que el trabajo ya fue completado?
- ¿Qué función cumple el visibility timeout y qué ocurre si el mensaje no se elimina?
- ¿Por qué el orden observado en una cola Standard no debe asumirse como estricto?
- ¿Qué riesgo permanece por ejecutar ambos procesos dentro de una sola EC2?

Conexión: S10 incorpora integración y procesamiento desacoplado. No se introducen SNS, Lambda, EventBridge avanzado, Kafka/MSK, Step Functions, Terraform ni observabilidad avanzada.

Fuentes oficiales AWS

- Amazon SQS standard queues — entrega at-least-once y orden best-effort.
- Amazon SQS visibility timeout — mensajes in-flight, reaparición y retry.
- ReceiveMessage API — WaitTimeSeconds y VisibilityTimeout.
- DeleteMessage API — eliminación mediante ReceiptHandle.
- Setting-up long polling in Amazon SQS — long polling con WaitTimeSeconds > 0.

- Boto3 credentials — credenciales de IAM role mediante EC2 instance metadata.
- Amazon EC2 security group rules — SSH restringido al origen requerido.